
Software Design Specification

<VoiceCoder AI>

Project Code: VC-WA01

Internal Advisor: Mr. Mudassar Ali Zaidi

External Advisor:

Project Manager: Dr. Muhammad Ilyas

Project Team: Alina Ahmed (S044)
Maryum Javed (S009)
Laiba Humayun (S031)

Submission Date: May 10,2026



Project Supervisors's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	VoiceCoder AI
Document	Software Design Specification
Document Version	2.0
Identifier	VCAI560
Status	Updated – Version 2.0
Author(s)	Alina Ahmed Maryum Javed Laiba Humayun
Approver(s)	PM
Issue Date	December 15, 2025
Document Location	
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

This section provides the definitions of all terms, acronyms, and abbreviations required to interpret the terms used in the document properly.

Term	Description
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DD	Design Specification
ERD	Entity Relationship Diagram
GPT-4o	Generative Pre-trained Transformer 4o (OpenAI)

JWT	JSON Web Token
NLP	Natural Language Processing
SRS	Software Requirements Specification
TTS	Text-to-Speech
UI	User Interface
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines

Table of Contents

1.Introduction.....	4
1.1 purpose of document	4
1.2 project overview.....	4
1.3 scope.....	5
2. Design Consideration.....	6
2.1 Assumptions and dependencies.....	6
2.2 Risk and volatile areas	6
3. System Architecture.....	7
3.1 System level architecture.....	7
3.2 Sub-system / Component level architecture.....	9
3.3 Sub-Component / Sub-Module level architecture.....	10
4. Design Strategies.....	13
4.1 Strategy 1...n.....	13
5. Detail System Design.....	15
5.1 Class Diagram.....	15
5.2 Sequence Diagram.....	18
5.3 State Transition Diagram.....	19
5.4 Logical Data Model (ERD).....	20
5.5 Physical Data Model.....	21
5.6 GUI Designs (Detailed).....	22
6.References.....	23

1. Introduction

1.1 Purpose of Document

This Design Document provides a comprehensive architectural blueprint and technical specification for the VoiceCoder AI web application. The document serves as a foundational reference for developers, stakeholders, and academic evaluators to understand the system's architecture, design decisions, and implementation strategy. It outlines the complete technical vision, ensuring consistent development across all team members and providing clear guidelines for future enhancements. This document is structured in alignment with IEEE Std 1016-2009 [20].

1.2 Project Overview

VoiceCoder AI is an innovative web-based application designed to enable programming through voice commands, specifically targeting the C++ programming language. The system converts spoken natural language into syntactically correct C++ code, providing an accessible programming environment for individuals with physical disabilities, repetitive strain injuries, or those seeking hands-free coding capabilities [4]. The application operates entirely within modern web browsers, requiring no local installation, and leverages cloud-based services for speech recognition, AI-powered code generation, compilation, and cloud file management.

Unlike existing tools such as Serenade and Speech2Code, VoiceCoder AI integrates GPT-4o AI for natural language code generation and debugging [12], provides Urdu Text-to-Speech support for Pakistani students [1], and offers a fully cloud-hosted, zero-cost platform with JWT-authenticated cloud file storage on Supabase PostgreSQL [7], [13].

Core Objectives:

- Provide voice-controlled C++ coding capability using Web Speech API [1]
- Create an inclusive programming environment for users with disabilities [4]
- Implement real-time speech-to-code conversion with noise gate filtering [1]
- Integrate GPT-4o AI for natural language code generation, debugging, and code review [12]
- Offer seamless cloud compilation via Judge0, Paiza.io, and JDoodle [2], [8]
- Support JWT-authenticated cloud file management with Supabase PostgreSQL [7], [13]
- Provide Urdu Text-to-Speech output using Speech Synthesis API [1]
- Maintain cross-browser compatibility on Chrome 70+ and Edge 79+ [1]

1.3 Scope

In-Scope Features:

- Voice-to-text conversion for C++ programming using Web Speech API [1]
- Three-level command parser: direct C++ snippets (Level 1), system commands (Level 2), and GPT-4o natural language processing (Level 3) [5], [12]
- Real-time code generation and editing via Monaco Editor with syntax highlighting [3]
- Cloud-based C++ compilation and execution via Judge0, Paiza.io, and JDoodle [2], [8]
- JWT authentication with bcrypt password hashing [7]
- PostgreSQL cloud database on Supabase for user files and AI chat history [13]
- Unique public share links for code files
- Urdu Text-to-Speech using Speech Synthesis API [1]
- Push-to-talk mode and custom voice command definitions
- Noise filtering using Web Audio API [1]
- Railway cloud hosting for the Node.js + Express backend [14]
- Responsive web UI for PC, laptop, and tablet

Out-of-Scope Features:

- Support for programming languages other than C++
- Advanced IDE features (debugging, version control, profiling)
- Offline functionality without internet connectivity
- Complex C++ features (templates, advanced metaprogramming)
- Mobile application development (iOS not supported)
- Real-time collaborative coding features

2. DESIGN CONSIDERATIONS

2.1 Assumptions and Dependencies

This system assumes users will run VoiceCoder AI on modern browsers supporting JavaScript and Web Speech API (Chrome 70+ or Edge 79+) [1]. Users are expected to speak clear English commands for voice recognition, with Urdu language available via the TTS output channel. A stable internet connection is required as the system depends on cloud APIs for compilation [2], [8] and AI code generation via GPT-4o [12]. It is further assumed that the user has a working microphone and basic knowledge of C++ syntax. Users must supply their own OpenAI API key to enable GPT-4o features [12].

Dependencies include:

- Web Speech API for real-time speech-to-text and Speech Synthesis API for Urdu TTS [1]
- Monaco Editor for the code editing interface [3]
- GPT-4o API (OpenAI) for natural language code generation and debugging [12]
- Online compiler APIs: Judge0, Paiza.io, and JDoodle [2], [8]
- Supabase PostgreSQL for cloud database and file management [13]
- Railway for Node.js + Express backend deployment [14]
- JWT and bcryptjs for authentication and password hashing [7]
- Network availability for all voice recognition, AI, and compilation tasks

2.2 Risks and Volatile Areas

The application is at risk if browser vendors change or deprecate the Web Speech API [1]. Environmental noise can distort voice recognition accuracy, which is partially mitigated by Web Audio API noise gate filtering. Compiler APIs may introduce rate limits or service downtimes [2], [8]. The GPT-4o API has associated token costs and is subject to rate limiting or policy changes by OpenAI [12].

Volatile areas include: API endpoint updates for compiler and AI services, expansion of the voice command dictionary [5], UI modifications based on future usability testing [4], and Supabase schema changes as new features are added [13].

3. System Architecture

3.1 System level architecture

Overall System View:

The VoiceCoder AI system follows a three-tier web application architecture with clear separation between presentation, business logic, and data processing layers. The architecture is designed for scalability, maintainability, and optimal user experience, following N-tier architecture best practices [22].

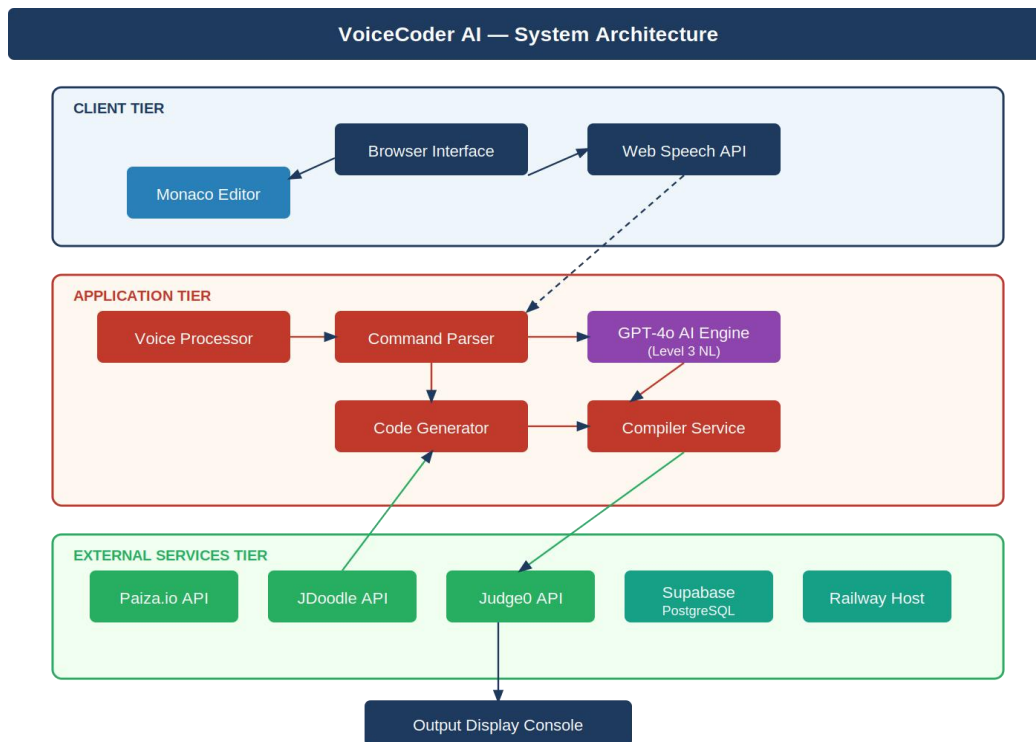


Figure 3.1: VoiceCoder AI System Architecture Diagram showing the three-tier structure with Client, Application, and External Service layers.

Component Interactions:

- User Interface Layer: HTML5, CSS3, JavaScript with Monaco Editor integration [3]
- Voice Processing Layer: Web Speech API for speech recognition and Web Audio API for noise filtering [1]
- Business Logic Layer: Three-level command parser, code generator, and GPT-4o integration for Level 3 natural language commands [5], [12]
- Authentication Layer: JWT tokens with bcrypt hashing; Supabase PostgreSQL for user data [7], [13]
- External Services Layer: Cloud compilation APIs (Judge0, Paiza.io, JDoodle) for code execution [2], [8]

- Cloud Infrastructure: Railway for backend hosting, Supabase for PostgreSQL database [13], [14]

Communication Protocols:

- HTTP/HTTPS: All client-server communications use HTTPS/TLS encryption [7]
- REST API: Integration with GPT-4o, compiler APIs, and Supabase [12], [13]
- JWT: Secure session management for all authenticated requests [7]
- WebSocket: Reserved for future real-time voice streaming enhancement

Deployment Architecture:

- Frontend: Static assets served via the Railway-hosted Node.js server [14]
- Backend: Node.js + Express server on Railway handling API proxying, JWT auth, and database calls [7], [14]
- Database: PostgreSQL on Supabase storing users, code files, and AI chat history [13]

Security Architecture:

- HTTPS enforcement for all communications [7]
- API keys for GPT-4o and compiler services stored server-side only — no frontend exposure [7]
- CORS policy implementation for API security
- Code execution in secure cloud sandboxes (Judge0, JDoodle, Paiza.io) [2], [8]

3.2 Sub-system / Module / Component level architecture

VoiceCoder AI is composed of multiple interacting components. The Voice Input Module captures microphone audio, which is forwarded to the Speech Recognition Component powered by Web Speech API [1]. A Web Audio API noise gate filters background noise before recognition. The Command Parser processes spoken text through a three-level command system: Level 1 (direct C++ templates), Level 2 (system commands such as compile, save, undo), and Level 3 (natural language via GPT-4o) [5], [12]. The Code Editor Component allows users to view, edit, and refine generated code using Monaco Editor [3]. The Compiler Integration Module sends code to cloud compiler APIs for execution [2], [8]. The Authentication and Storage Module manages user sessions, file saves, and chat history via Supabase [13]. The Urdu TTS module reads output back to users [1].

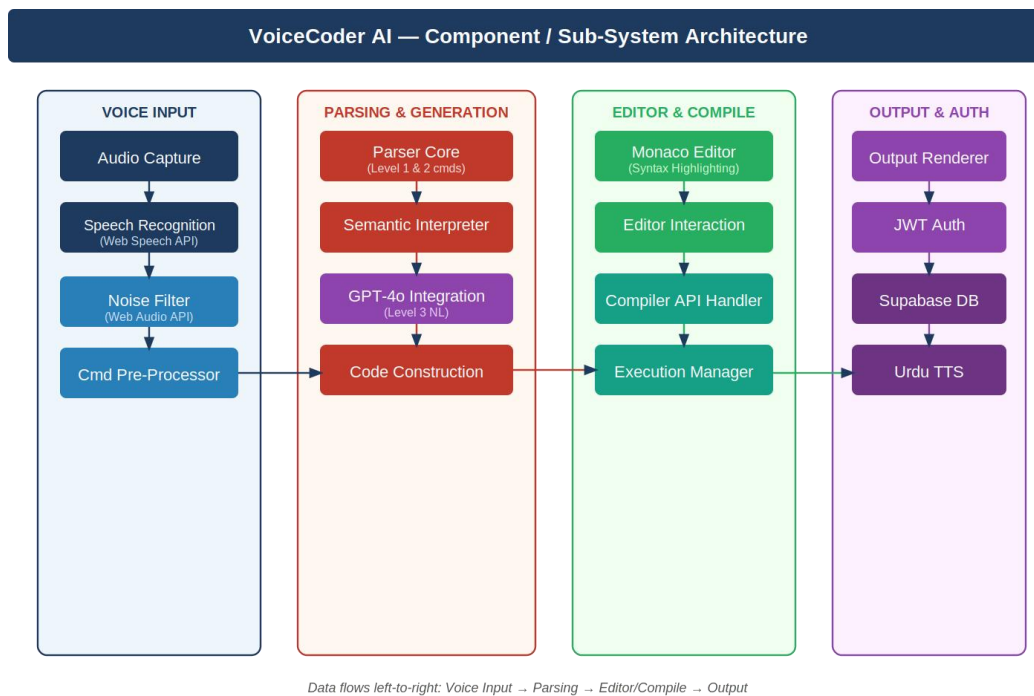


Figure 3.2: VoiceCoder AI Component/Sub-System Architecture Diagram showing data flow from Voice Input through Parsing and Generation to Compilation and Output.

3.3 Sub-Component/ Sub-Module level architecture

The VoiceCoder AI system is decomposed into granular sub-modules to ensure modularity, maintainability, and extensibility [1], [2]. Each sub-module performs a clearly defined responsibility, interacting through well-structured interfaces. This enables efficient development and easier future upgrades without impacting the entire system.

A. Speech Processing Sub-Modules

These modules together manage the voice input pipeline:

- 1. Audio Capture Module:** Captures real-time audio through the browser microphone. Applies noise gate filtering using Web Audio API. Sends audio to the Web Speech API for transcription [1].
- 2. Speech Recognition Engine (Web Speech API Wrapper):** Converts audio into text streams. Handles recognition events (start, end, error, result). Returns recognized phrases for parsing [1].
- 3. Command Pre-Processor:** Cleans, normalizes, and tokenizes recognized text. Handles variations such as “for loop”, “four loop”, and “create for loop” all mapping to the same command.

B. Parsing & Code Generation Sub-Modules

- 1. Parser Core:** Maps processed text to C++ syntax templates. Uses rule-based command dictionaries (Level 1 and Level 2) drawn from SRS-defined C++ constructs [5]. Ensures syntactic correctness by validating commands.
- 2. Semantic Interpreter:** Handles variable assignments, conditions, and iterative ranges. For example: “for i from 0 to 10” detects variable=i, start=0, end=10, and generates the appropriate C++ for loop.
- 3. GPT-4o Natural Language Integration:** Handles Level 3 natural language commands by sending user requests to the GPT-4o API and receiving complete C++ code blocks in return [12]. Supports code generation, error fixing, code explanation, and code review.
- 4. Code Construction Engine:** Builds formatted C++ blocks. Maintains indentation and scope levels. Appends generated code to the Monaco Editor [3].

C. Coding Interface Sub-Modules

- 1. Monaco Rendering Module:** Displays generated C++ code with syntax highlighting, indentation management, and cursor management [3]. Supports user edits before compilation.
- 2. Editor Interaction Layer:** Bridges voice-generated code and editor operations. Handles undo/redo, line insertion, block removal, and push-to-talk mode toggling.

D. Compilation & Output Sub-Modules

- 1. Compiler API Handler:** Sends C++ code to Paiza.io, Judge0, or JDoodle via secure HTTPS requests. Server-side proxy on Railway protects API keys [2], [7], [8].
- 2. Execution Manager:** Waits for API response. Handles runtime errors, compiler errors, and infinite loops safely within the cloud sandbox [2], [8].
- 3. Output Renderer:** Displays program results in the output console. Formats warnings, errors, and runtime responses. Triggers Urdu TTS for spoken output [1].

E. Authentication & Storage Sub-Modules

- 1. JWT Authentication Module:** Handles user registration, login, and session management using JWT tokens and bcrypt password hashing [7].

2. Supabase Storage Manager: Manages CRUD operations for code files, AI chat history, and user preferences in PostgreSQL [13].

3. Share Link Generator: Creates unique public URLs for code files enabling sharing without requiring login.

F. System Utility Sub-Modules

1. Error Logging & Handling Module: Captures parser errors, recognition issues, API failures, and network errors. Displays user-friendly messages with retry guidance.

2. Session Manager: Maintains temporary in-session data (recognized commands, logs). Clears sensitive data securely at session end [7].

4. Design Strategies

The design of VoiceCoder AI follows strategic decisions to enhance system quality attributes such as performance, usability, scalability, security, and accessibility [4], [16].

4.1 Strategy 1 : Modular, Component-Based Architecture

Reasoning:

The system integrates several external technologies (Web Speech API, Monaco Editor, GPT-4o API, compiler APIs, Supabase) which may evolve over time [12], [13]. A modular design isolates each subsystem, making future replacement or updates straightforward without impacting the entire codebase.

Trade-offs:

Higher initial planning time, but long-term flexibility and reduced regression risks during API updates.

4.2 Strategy 2 : Event-Driven User Interface

Reasoning:

Speech recognition, AI code generation, parsing, and compilation are all asynchronous operations. An event-driven model ensures smooth handling of recognition events, GPT-4o API callbacks, and editor updates [12].

Benefits:

- Zero UI freezing during long operations
- Real-time feedback via waveform animation
- Clean separation of UI and logic

4.3 Strategy 3 : Three-Level Command Parsing

Reasoning:

Rather than relying solely on NLP models, a three-level tiered approach ensures predictable, deterministic code generation for academic C++ constructs (Level 1 and Level 2), while still providing GPT-4o power for complex natural language requests (Level 3) [5], [12].

Benefits:

- High accuracy for academic C++ commands at Level 1/2
- Unlimited expressiveness via AI for complex requests at Level 3
- Easy to expand with new command rules
- Reduced API token costs for simple operations

4.4 Strategy 4 : Cloud-Sandboxed Execution

Reasoning:

User-generated C++ code can be unsafe or malformed. Running it in secure cloud sandboxes (Judge0, JDoodle, Paiza.io) prevents system-level threats and ensures zero local machine access [2], [8].

Benefits:

- No local machine access risk
- Safe handling of malformed or potentially harmful code
- Zero installation required

4.5 Strategy 5 : Security-First Backend Design

Reasoning:

Following OWASP Top Ten guidelines [7], all API keys are stored server-side on Railway, JWT tokens secure all authenticated sessions, passwords are bcrypt-hashed, and all communications use HTTPS/TLS.

Design choices:

- Server-side API key proxying — no keys in frontend code [7]
- JWT session security with configurable expiry [7]
- bcrypt password hashing with salt rounds [7]

4.6 Strategy 6 : Accessibility-First UI Design

Reasoning:

Target users include individuals with RSI and physical disabilities [4]. The interface is designed to be fully operable via voice with minimal keyboard dependency, following WCAG 2.1 guidelines [16].

Design choices:

- Large buttons (Start Recording, Stop, Compile, Save, Share)
- High-contrast Monaco Editor dark theme
- Voice-driven actions with minimal manual input
- Urdu TTS for full output readback [1]

5. Detail System Design

5.1 Class Diagram

Core Classes Structure:

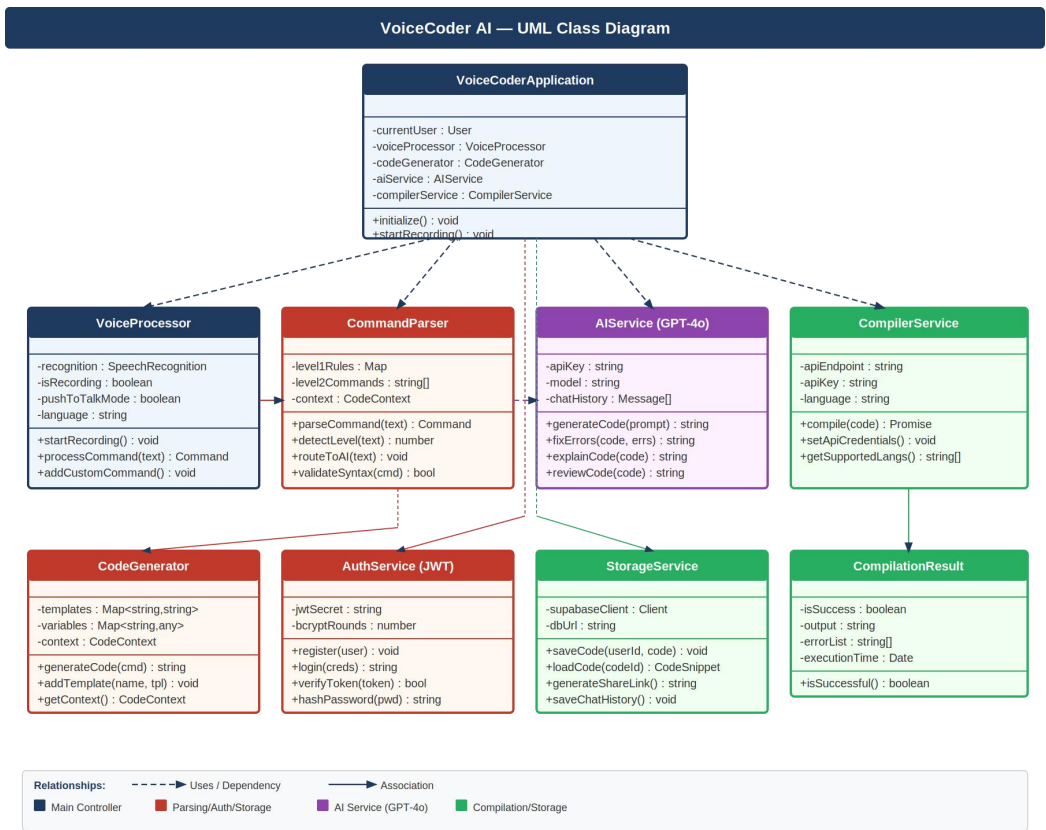


Figure 5.1: UML Class Diagram illustrating core classes, their attributes, methods, and relationships in the VoiceCoder AI application.

Class Descriptions:

1. VoiceCoderApplication (Main Controller Class)

Purpose: Central coordinator managing all system components.

Attributes: currentUser, voiceProcessor, codeGenerator, aiService (GPT-4o), compilerService, authService, storageService, uiController.

Key Methods: initialize(), startRecording(), stopRecording(), compileCode(), saveCode(), shareCode().

2. VoiceProcessor (Voice Recognition Handler)

Purpose: Processes voice input and converts to commands using Web Speech API [1].

Attributes: recognition (Web Speech API instance), isRecording, pushToTalkMode, language, commands (Map).

Key Methods: startRecording(), stopRecording(), processCommand(text) → Command, addCustomCommand(), setLanguage().

3. CommandParser (Three-Level Parser)

Purpose: Routes voice commands through the three-level parsing system [5].

Attributes: level1Rules (Map), level2Commands (string[]), context (CodeContext).

Key Methods: parseCommand(text), detectLevel(text) → 1|2|3, routeToAI(text), validateSyntax(cmd) → boolean.

4. AIService — GPT-4o Integration

Purpose: Manages GPT-4o API calls for natural language code generation, debugging, explanation, and review [12].

Attributes: apiKey, model (string), chatHistory (Message[]).

Key Methods: generateCode(prompt) → string, fixErrors(code, errors) → string, explainCode(code) → string, reviewCode(code) → string.

5. CodeGenerator (Code Construction Engine)

Purpose: Generates C++ code from parsed Level 1 and Level 2 commands [5].

Attributes: templates (Map<string,string>), variables (Map<string,any>), context (CodeContext).

Key Methods: generateCode(command) → string, addTemplate(name, template), getContext() → CodeContext.

6. CompilerService (Compilation Manager)

Purpose: Handles code compilation through external APIs [2], [8].

Attributes: apiEndpoint, apiKey, language, version.

Key Methods: compile(code) → Promise<CompilationResult>, setApiCredentials(), getSupportedLanguages() → string[].

7. AuthService (JWT Authentication)

Purpose: Manages registration, login, and session tokens using JWT and bcrypt [7].

Attributes: jwtSecret, bcryptRounds.

Key Methods: register(user), login(credentials) → string, verifyToken(token) → boolean, hashPassword(password).

8. StorageService (Supabase Manager)

Purpose: CRUD operations for code files, chat history, and preferences in Supabase PostgreSQL [13].

Key Methods: saveCode(userId, code), loadCode(codeId) → CodeSnippet, deleteCode(codeId), generateShareLink(codeId) → string, saveChatHistory(userId, chat).

Supporting Classes:

Command (voice command with level 1/2/3 and parameters), CodeContext (programming context with variables/includes/scope), CompilationResult (output and error list), CodeSnippet (saved code with shareLink metadata), UserPreferences (theme, voice speed, language, autoCompile setting).

Class Relationships:

- Aggregation: VoiceCoderApplication contains VoiceProcessor, CodeGenerator, AIService, CompilerService, AuthService, and StorageService
- Composition: CodeGenerator owns CodeContext
- Association: VoiceProcessor uses Command objects; CommandParser routes to AIService for Level 3
- Dependency: CompilerService depends on external API configuration [2], [8]
- Dependency: AIService depends on GPT-4o API key and endpoint [12]

Design Patterns Implemented:

- Factory Pattern: Command creation based on detected voice input level
- Strategy Pattern: Different code generation strategies for Level 1, Level 2, and Level 3 AI commands [12]
- Observer Pattern: UI updates triggered by voice recognition events and API responses
- Template Method Pattern: Code generation with extensible C++ templates [5]
- Singleton Pattern: Application configuration and API key management [7]
- Proxy Pattern: Server-side API key proxying for security [7]

Data Structures:

- HashMap: Command lookup and template storage ($O(1)$ average access)
- Stack: Managing C++ code scope and indentation levels (LIFO operations)
- Queue: Processing voice command sequences (FIFO operations)
- Trie: $O(n)$ command prefix matching where n is command length
- LRU Cache: Frequently used code template caching

5.2 SEQUENCE DIAGRAM

Below is the conceptual flow of how the system handles voice-driven code generation in VoiceCoder AI, including the three-level command routing, GPT-4o AI integration, cloud compilation, and Urdu TTS output [1], [2], [12].

The user speaks a command, which is captured as audio. This audio is converted into text by the Web Speech API [1]. The Command Parser analyzes this text: if Level 1 or Level 2, the Code Generator builds the C++ block directly [5]; if Level 3, the request is sent to the GPT-4o API which returns generated C++ code [12]. When the user triggers “Compile”, the code is transmitted to a cloud compiler API (Paiza.io, Judge0, or JDoodle) [2], [8], which returns the output. Results are displayed in the console and read aloud via Urdu TTS [1]. The user may then save or share the file via JWT-authenticated Supabase storage [7], [13].

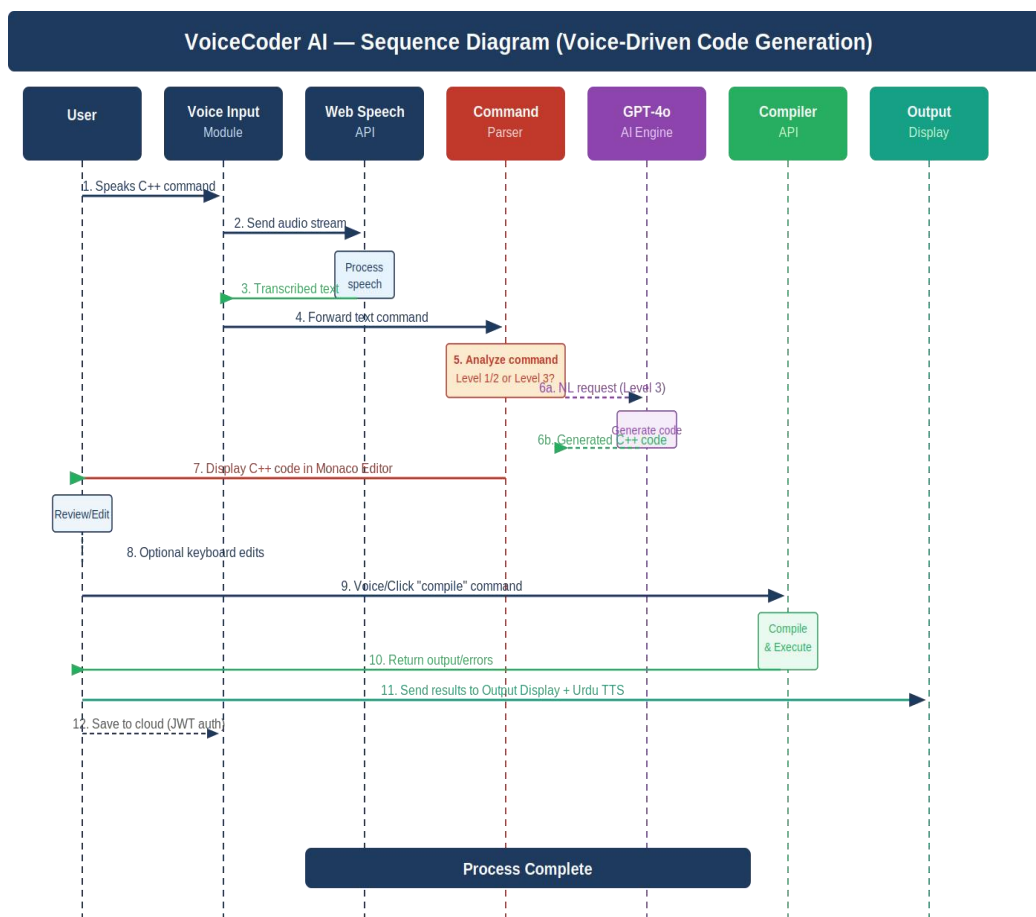


Figure 5.2: VoiceCoder AI Sequence Diagram — Voice-Based Code Generation Flow showing interactions between User, Voice Input Module, Web Speech API, Command Parser, GPT-4o AI Engine, Compiler API, and Output Display.

5.3 State Transition Diagram

Primary States

1. Idle
2. Listening
3. Processing Speech
4. Parsing Command
- 4b. AI Generation (GPT-4o — Level 3 requests)
5. Generating Code
6. Ready to Compile
7. Compiling
8. Displaying Output

Transitions occur based on voice events, parsing success, GPT-4o responses, or compiler API callbacks.

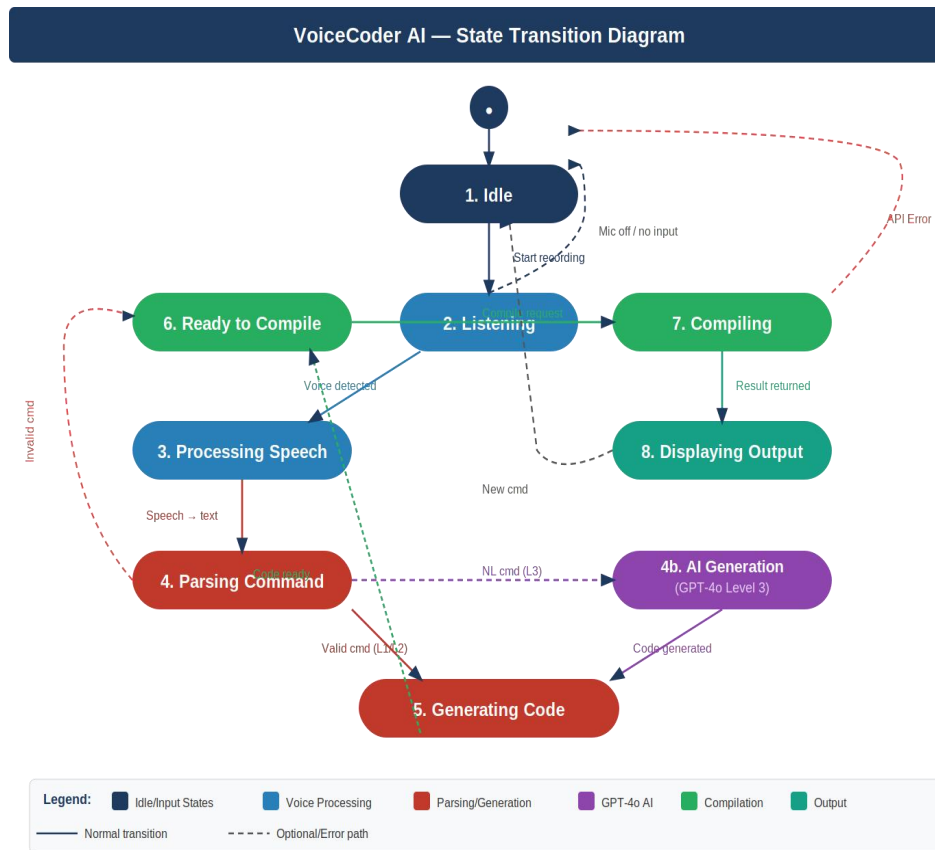


Figure 5.3: VoiceCoder AI State Transition Diagram showing all eight primary states and transition paths including the GPT-4o AI Generation branch.

5.4 Logical Data Model (ERD)

Entities & Attributes:

User

- userId (PK)
- username, email
- passwordHash (bcrypt) [7]
- jwtToken, createdAt
- preferences (JSON), language (Urdu/EN)

UserSession

- sessionId (PK)
- userId (FK)
- startTime, endTime
- microphoneStatus, browserInfo

VoiceCommand

- commandId (PK)
- sessionId (FK)
- commandText, commandLevel (1/2/3)
- timestamp, parsedStructure
- aiPrompt (for Level 3 GPT-4o commands)

GeneratedCode

- codeId (PK)
- commandId (FK), userId (FK)
- cppBlock (text), indentationLevel, lineNumber
- shareLink (unique public URL)

CompilationResult

- resultId (PK)
- codeId (FK)
- compilerUsed (Judge0 / Paiza.io / JDoodle) [2], [8]
- output (text), errorList (JSON)
- executionTime, compiledAt

ChatHistory (AI Conversations)

- chatId (PK), userId (FK)
- userPrompt, aiResponse (GPT-4o) [12]
- timestamp, tokens_used

Relationships:

- One User → many UserSessions (1:N)
- One UserSession → many VoiceCommands (1:N)
- One VoiceCommand → one GeneratedCode block (1:1)
- One GeneratedCode → many CompilationResults (1:N, multiple compile runs)
- One User → many ChatHistory records (1:N, stored AI conversations) [12]

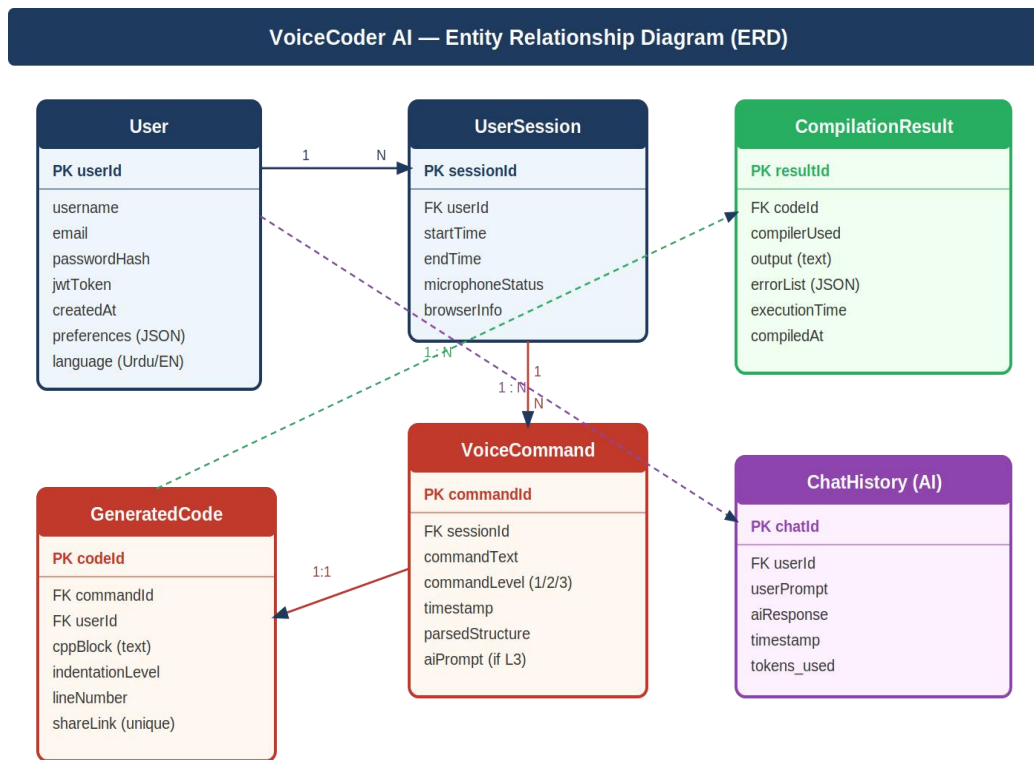


Figure 5.4: VoiceCoder AI Entity Relationship Diagram showing User, UserSession, VoiceCommand, GeneratedCode, CompilationResult, and ChatHistory (AI) entities with their relationships.

5.5 Physical Data Model

VoiceCoder AI uses a hybrid storage architecture. Persistent data is stored in Supabase PostgreSQL [13]. Temporary in-session data is kept in browser memory objects. No sensitive data is stored in browser LocalStorage or SessionStorage, in compliance with OWASP security guidelines [7].

- Supabase PostgreSQL: Users, UserSessions, GeneratedCode, CompilationResults, ChatHistory tables [13]
- In-memory Objects: Commands, template cache, and parsing results during active session
- Server-side environment variables (Railway): API keys for GPT-4o [12], compiler APIs [2], [8], and Supabase credentials [13]
- External API Storage not used persistently — compiler APIs return output only [2], [8]

No sensitive credentials or API keys are exposed in the frontend. All cloud storage access is mediated through the Railway backend with JWT-authenticated requests [7], [14].

5.6 GUI Designs (Detailed)

1. Home / Login Screen

VoiceCoder AI title banner with animated waveform logo. Login and Register forms with email and password fields. JWT-secured login flow [7].

2. Main Dashboard

Large microphone button (Start Recording / Push-to-Talk). Live waveform animation during speech capture. Voice status indicator (Idle / Listening / Processing). Quick command reference guide button.

3. Code Editor Screen

Monaco Editor with:

- Dark theme
- Syntax colors and line numbers
- Auto indentation
- Multi-tab support for multiple code files

Buttons:

- Run Code
- Stop Recording
- Clear Code
- Save to Cloud (JWT authenticated) [7], [13]
- Share — generates unique public link
- Open Command Guide
- AI Chat Panel for GPT-4o interactions [12]

4. Output Console

Panel below editor:

- Program output, warnings, compilation errors
- Auto-scroll enabled

- Urdu TTS toggle for spoken output readback [1]

5. Error Popup Modals

Invalid command notifications, API timeout messages, microphone access alerts, and authentication error dialogs.

6.References

- [1] Mozilla Developer Network (MDN), “Web Speech API,” Mozilla Foundation, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- [2] Paiza, Inc., “Paiza.IO Online Compiler API Documentation,” Paiza, 2024. [Online]. Available: <https://paiza.io/en/api>
- [3] Microsoft, “Monaco Editor Documentation,” Microsoft Docs, 2024. [Online]. Available: <https://microsoft.github.io/monaco-editor/docs.html>
- [4] World Wide Web Consortium (W3C), “Web Accessibility Initiative (WAI),” 2024. [Online]. Available: <https://www.w3.org/WAI/>
- [5] cppreference.com, “C++ Language Reference,” 2024. [Online]. Available: <https://cppreference.com>
- [6] Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner’s Approach, 8th ed. New York: McGraw-Hill, 2015.
- [7] Open Web Application Security Project (OWASP), “OWASP Top Ten,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [8] JDoodle, “JDoodle Compiler API Documentation,” 2024. [Online]. Available: <https://www.jdoodle.com/compiler-api>
- [9] Atlassian, “Iterative and Incremental Development,” 2024. [Online]. Available: <https://www.atlassian.com/project-management>
- [10] Gamma, E., Helm, R., Johnson, R., and Vlissides, J., Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley, 1994.
- [11] Jurafsky, D. and Martin, J. H., Speech and Language Processing, 3rd ed. Upper Saddle River: Pearson, 2023.
- [12] OpenAI, “GPT-4o API Documentation,” 2024. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o>
- [13] Supabase, “Open Source Firebase Alternative with PostgreSQL,” 2024. [Online]. Available: <https://supabase.com/docs>
- [14] Railway, “Cloud Application Hosting Platform,” 2024. [Online]. Available: <https://docs.railway.app>
- [15] Nielsen, J., Usability Engineering. San Francisco: Morgan Kaufmann, 1994.
- [16] W3C, Web Content Accessibility Guidelines (WCAG) 2.1, W3C Recommendation, 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/>
- [17] Object Management Group (OMG), Unified Modeling Language (UML) Specification, Version 2.5.1, 2017.
- [18] Sommerville, I., Software Engineering, 10th ed. Boston: Pearson Education, 2016.
- [19] Silberschatz, A., Galvin, P. B., and Gagne, G., Operating System Concepts, 10th ed. Hoboken: Wiley, 2018.
- [20] IEEE, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions, IEEE Std 1016-2009, 2009.
- [21] Stroustrup, B., The C++ Programming Language, 4th ed. Boston: Addison-Wesley, 2013.
- [22] Microsoft Corporation, “N-tier architecture style,” Microsoft Azure Architecture Guide, 2023.
- [23] World Health Organization, Global report on assistive technology, WHO, 2022. [Online]. Available: <https://www.who.int/publications/i/item/9789240049451>